

✓ Pregunta 1: Optimización de Ruta para Vehículos Autónomos con Búsqueda

Una empresa de logística está desarrollando un sistema de vehículos autónomos para optimizar la entrega de paquetes en una ciudad. La ciudad puede ser modelada como un grafo donde los nodos son intersecciones y los aristas son calles con un costo asociado (tiempo de viaje). Para evitar congestiones y accidentes, el vehículo debe navegar desde un punto de inicio S hasta un destino D .

Objetivo: Desarrolle un pseudocódigo para un algoritmo de búsqueda A^* que permita al vehículo autónomo encontrar la ruta óptima. Considere que tiene acceso a una función heurística $h(n)$ que estima la distancia en línea recta (o euclidiana) desde cualquier nodo n hasta el destino D .

Consideraciones:

- El grafo es no dirigido y los costos de los aristas son positivos.
- El algoritmo debe mantener un registro del costo acumulado desde el inicio $g(n)$ y una estimación del costo total $f(n) = g(n) + h(n)$.
- Debe manejar una cola de prioridad para explorar los nodos de manera eficiente.
- Asuma que las estructuras de datos básicas (listas, diccionarios, colas de prioridad) están disponibles.

✓ Solución

Algoritmo A_Estrella_Vehiculo_Autonomo(Grafo G, Nodo Inicio S, Nodo Destino D, Heuristica h)

// G: Grafo de la ciudad con nodos y aristas ponderados

// S: Nodo de inicio

// D: Nodo de destino

// h: Función heurística $h(n)$ que estima el costo de n a D

// Inicializar estructuras de datos

abiertos = ColaPrioridad() // Almacena nodos a explorar, ordenados por $f(n)$

cerrados = Conjunto() // Almacena nodos ya explorados

costo_g = Diccionario() // $costo_g[n]$ = costo desde S hasta n

padres = Diccionario() // $padres[n]$ = nodo previo en la ruta más corta conocida a n

// Inicializar el nodo de inicio

$costo_g[S] = 0$

$abiertos.insertar(S, h(S))$ // (Nodo, Prioridad $f(S)$)

Mientras abiertos no esté vacía:

$actual = abiertos.extraer_min()$ // Extrae el nodo con el menor $f(n)$

 Si $actual$ es D :

 // Se encontró el destino, reconstruir y retornar la ruta

 Ruta = []

 temp = actual

Mientras temp no es NULO:

Ruta.añadir_al_frente(temp)

temp = padres[temp]

Retornar Ruta

Añadir actual a cerrados

Para cada vecino v de actual en G:

Si v está en cerrados:

Continuar // Ya explorado

tentativo_g = costo_g[actual] + costo_arista(actual, v)

Si v no está en abiertos O tentativo_g < costo_g[v]:

// Se encontró un camino mejor o nuevo

padres[v] = actual

costo_g[v] = tentativo_g

costo_f = tentativo_g + h(v)

Si v no está en abiertos:

abiertos.insertar(v, costo_f)

Si tentativo_g < costo_g[v]: // Solo si se actualizó un nodo existente en abiertos

abiertos.actualizar_prioridad(v, costo_f)

Retornar "No se encontró ruta" // No hay camino de S a D

▼ Pregunta 2: Descenso de Gradiente Estocástico con Momento y Regu

Considere una red neuronal *feedforward* con una capa de entrada, una única capa oculta con H neuronas, y una capa de salida con K neuronas. Asuma que todas las neuronas usan la función de activación **ReLU** ($f(x) = \max(0, x)$) en las capas ocultas y de salida, excepto que la capa de salida final utiliza una función de activación **Softmax** para problemas de clasificación multiclase. La función de pérdida es la **entropía cruzada categórica**.

Dado un conjunto de datos de entrenamiento $\{(x^{(m)}, t^{(m)})\}_{m=1}^M$, donde $x^{(m)} \in \mathbb{R}^D$ es el vector de entrada y $t^{(m)} \in \{0, 1\}^K$ es el vector *one-hot* del objetivo para la m -ésima muestra.

La función de pérdida para una única muestra m es:

$$E^{(m)} = - \sum_{k=1}^K t_k^{(m)} \log(y_k^{(L,m)})$$

Donde $y_k^{(L,m)}$ es la activación de la k -ésima neurona de salida para la muestra m .

Se desea entrenar esta red utilizando **Descenso de Gradiente Estocástico (SGD)** con **momento** y **regularización L2 (weight decay)**.

a) **Derivación de Backpropagation con ReLU y Softmax-CrossEntropy**: Deduzca las ecuaciones completas para las gradientes de los pesos y biases utilizando el algoritmo de backpropagation para esta

arquitectura específica. Preste especial atención a la derivada de la función ReLU y a la combinación

Softmax-CrossEntropy (demuestre que $\frac{\partial E^{(m)}}{\partial \text{net}_k^{(L,m)}} = y_k^{(L,m)} - t_k^{(m)}$).

b) **Regla de Actualización con Momento y Regularización L2:** Formule las reglas de actualización para los pesos $W^{(l)}$ y biases $b^{(l)}$ de la red, incorporando los términos de **momento** y **regularización L2**. Sea η la tasa de aprendizaje, γ el coeficiente de momento y λ el coeficiente de regularización L2. Defina claramente todas las variables involucradas en sus ecuaciones de actualización.

▼ Solución

a) Derivación de Backpropagation con ReLU y Softmax-CrossEntropy

Asumimos una red con:

- Capa de entrada: $x \in \mathbb{R}^D$
- Capa oculta: $h \in \mathbb{R}^H$
- Capa de salida: $y \in \mathbb{R}^K$

Notación:

- $z_j^{(l)}$: entrada neta (pre-activación) de la neurona j en la capa l .
- $a_j^{(l)}$: activación de la neurona j en la capa l .
- $W^{(l)}$: matriz de pesos de la capa l (conectando capa $l - 1$ a capa l).
- $b^{(l)}$: vector de biases de la capa l .
- $t^{(m)}$: vector *one-hot* objetivo para la muestra m .
- $y^{(L,m)}$: salida de la red para la muestra m .

Pase Hacia Adelante (Forward Pass): Para una capa l (donde $l = 1$ es la capa oculta, $l = L = 2$ es la capa de salida):

1. **Capa Oculta (l=1):** $z_j^{(1,m)} = \sum_{i=1}^D W_{ji}^{(1)} x_i^{(m)} + b_j^{(1)} a_j^{(1,m)} = \text{ReLU}(z_j^{(1,m)}) = \max(0, z_j^{(1,m)})$

2. **Capa de Salida (l=2=L):** $z_k^{(L,m)} = \sum_{j=1}^H W_{kj}^{(L)} a_j^{(1,m)} + b_k^{(L)}$

$$y_k^{(L,m)} = \text{Softmax}(z_k^{(L,m)}) = \frac{e^{z_k^{(L,m)}}}{\sum_{p=1}^K e^{z_p^{(L,m)}}}$$

Función de Pérdida (Cross-Entropy para una muestra m):

$$E^{(m)} = - \sum_{k=1}^K t_k^{(m)} \log(y_k^{(L,m)})$$

Pase Hacia Atrás (Backward Pass):

1. **Gradiente de la pérdida con respecto a las entradas netas de la capa de salida ($\delta^{(L,m)}$):** Necesitamos calcular $\frac{\partial E^{(m)}}{\partial z_k^{(L,m)}}$. Sabemos que $E^{(m)} = - \sum_{k'=1}^K t_{k'}^{(m)} \log(y_{k'}^{(L,m)})$. Consideremos

$$\frac{\partial E^{(m)}}{\partial z_k^{(L,m)}} = \sum_{k'=1}^K t_{k'}^{(m)} \frac{\partial E^{(m)}}{\partial y_{k'}^{(L,m)}} \frac{\partial y_{k'}^{(L,m)}}{\partial z_k^{(L,m)}}.$$

La derivada de Softmax es: $\frac{\partial y_{k'}^{(L,m)}}{\partial z_k^{(L,m)}} = \begin{cases} y_k^{(L,m)}(1 - y_k^{(L,m)}) & \text{si } k' = k \\ -y_{k'}^{(L,m)} y_k^{(L,m)} & \text{si } k' \neq k \end{cases}$

La derivada de la entropía cruzada con respecto a $y_{k'}^{(L,m)}$ es: $\frac{\partial E^{(m)}}{\partial y_{k'}^{(L,m)}} = -\frac{t_{k'}^{(m)}}{y_{k'}^{(L,m)}}$

Sustituyendo y simplificando (esta es una derivación estándar y crucial):

$$\frac{\partial E^{(m)}}{\partial z_k^{(L,m)}} = \left(-\frac{t_k^{(m)}}{y_k^{(L,m)}} \right) y_k^{(L,m)}(1 - y_k^{(L,m)}) + \sum_{k' \neq k} \left(-\frac{t_{k'}^{(m)}}{y_{k'}^{(L,m)}} \right) (-y_{k'}^{(L,m)} y_k^{(L,m)})$$

$$\frac{\partial E^{(m)}}{\partial z_k^{(L,m)}} = -t_k^{(m)}(1 - y_k^{(L,m)}) + \sum_{k' \neq k} t_{k'}^{(m)} y_k^{(L,m)}$$

$\frac{\partial E^{(m)}}{\partial z_k^{(L,m)}} = -t_k^{(m)} + t_k^{(m)} y_k^{(L,m)} + y_k^{(L,m)} \sum_{k' \neq k} t_{k'}^{(m)}$ Dado que $t^{(m)}$ es un vector *one-hot*, solo un $t_k^{(m)}$ es 1 y el resto son 0. Si $t_k^{(m)} = 1$, entonces $\sum_{k' \neq k} t_{k'}^{(m)} = 0$. $\frac{\partial E^{(m)}}{\partial z_k^{(L,m)}} = -1 + y_k^{(L,m)}$. Si $t_k^{(m)} = 0$, entonces existe algún $t_j^{(m)} = 1$ donde $j \neq k$. $\frac{\partial E^{(m)}}{\partial z_k^{(L,m)}} = 0 + y_k^{(L,m)}(1) = y_k^{(L,m)}$. En ambos casos, la combinación

Softmax-CrossEntropy simplifica a:

$$\delta_k^{(L,m)} = \frac{\partial E^{(m)}}{\partial z_k^{(L,m)}} = y_k^{(L,m)} - t_k^{(m)}$$

2. Gradientes de pesos y biases de la capa de salida (L):

$$\frac{\partial E^{(m)}}{\partial W_{kj}^{(L)}} = \delta_k^{(L,m)} a_j^{(1,m)}$$

$$\frac{\partial E^{(m)}}{\partial b_k^{(L)}} = \delta_k^{(L,m)}$$

3. Propagación de δ a la capa oculta ($l = 1$): $\delta_j^{(1,m)} = \frac{\partial E^{(m)}}{\partial z_j^{(1,m)}} = \sum_{k=1}^K \frac{\partial E^{(m)}}{\partial z_k^{(L,m)}} \frac{\partial z_k^{(L,m)}}{\partial a_j^{(1,m)}} \frac{\partial a_j^{(1,m)}}{\partial z_j^{(1,m)}}$

$$\delta_j^{(1,m)} = \left(\sum_{k=1}^K \delta_k^{(L,m)} W_{kj}^{(L)} \right) \cdot \frac{\partial \text{ReLU}(z_j^{(1,m)})}{\partial z_j^{(1,m)}} \text{ La derivada de ReLU es: } \frac{\partial \text{ReLU}(x)}{\partial x} = \begin{cases} 1 & \text{si } x > 0 \\ 0 & \text{si } x \leq 0 \end{cases} \text{ (o}$$

indefinida en 0, pero se asume 0 o 1). Entonces:

$$\delta_j^{(1,m)} = \left(\sum_{k=1}^K \delta_k^{(L,m)} W_{kj}^{(L)} \right) \cdot \mathbb{I}(z_j^{(1,m)} > 0)$$

Donde $\mathbb{I}(\cdot)$ es la función indicadora.

4. Gradientes de pesos y biases de la capa oculta (1):

$$\frac{\partial E^{(m)}}{\partial W_{ji}^{(1)}} = \delta_j^{(1,m)} x_i^{(m)}$$

$$\frac{\partial E^{(m)}}{\partial b_j^{(1)}} = \delta_j^{(1,m)}$$

b) Regla de Actualización con Momento y Regularización L2

El costo total para una muestra m con regularización L2 es:

$$E_{total}^{(m)} = E^{(m)} + \frac{\lambda}{2} \left(\sum_{l=1}^L \|W^{(l)}\|_F^2 \right)$$

Donde $\|W^{(l)}\|_F^2 = \sum_{j,i} (W_{ji}^{(l)})^2$ es la norma Frobenius cuadrada de la matriz de pesos.

La gradiente del término de regularización con respecto a un peso $W_{ji}^{(l)}$ es $\lambda W_{ji}^{(l)}$. Entonces, el gradiente total que usamos para la actualización es:

$$\frac{\partial E_{total}^{(m)}}{\partial W_{ji}^{(l)}} = \frac{\partial E^{(m)}}{\partial W_{ji}^{(l)}} + \lambda W_{ji}^{(l)}$$

Para los biases, no se aplica regularización L2 típicamente.

Reglas de Actualización (SGD con Momento y Regularización L2):

Primero, definimos las variables de momento (velocidades) $V_W^{(l)}$ y $V_b^{(l)}$ que se inicializan a cero.

Para cada peso $W_{ji}^{(l)}$ en cada capa l :

- Cálculo de la gradiente del peso para la muestra actual:** $g_{W_{ji}^{(l)}} = \frac{\partial E^{(m)}}{\partial W_{ji}^{(l)}} + \lambda W_{ji}^{(l)}$ (Donde $\frac{\partial E^{(m)}}{\partial W_{ji}^{(l)}}$ es la gradiente obtenida en la parte a).
- Actualización de la velocidad (momento):** $V_{W_{ji}^{(l)}} \leftarrow \gamma V_{W_{ji}^{(l)}} + \eta g_{W_{ji}^{(l)}}$
- Actualización del peso:** $W_{ji}^{(l)} \leftarrow W_{ji}^{(l)} - V_{W_{ji}^{(l)}}$

Para cada bias $b_j^{(l)}$ en cada capa l :

- Cálculo de la gradiente del bias para la muestra actual:** $g_{b_j^{(l)}} = \frac{\partial E^{(m)}}{\partial b_j^{(l)}}$ (Donde $\frac{\partial E^{(m)}}{\partial b_j^{(l)}}$ es la gradiente obtenida en la parte a).
- Actualización de la velocidad (momento):** $V_{b_j^{(l)}} \leftarrow \gamma V_{b_j^{(l)}} + \eta g_{b_j^{(l)}}$
- Actualización del bias:** $b_j^{(l)} \leftarrow b_j^{(l)} - V_{b_j^{(l)}}$

Variables:

- η : Tasa de aprendizaje (escalar positivo). Controla el tamaño del paso en la dirección del gradiente.
- γ : Coeficiente de momento (escalar, $0 \leq \gamma < 1$). Determina cuánto contribuye la "velocidad" anterior a la actualización actual, ayudando a acelerar el descenso en direcciones consistentes y amortiguar oscilaciones.
- λ : Coeficiente de regularización L2 (escalar no negativo). Penaliza pesos grandes para prevenir sobreajuste.
- $g_{W_{ji}^{(l)}}, g_{b_j^{(l)}}$: Gradientes del costo con respecto a los pesos y biases, respectivamente, para la muestra actual.
- $V_{W_{ji}^{(l)}}, V_{b_j^{(l)}}$: Variables de velocidad (momento) para los pesos y biases, respectivamente.

✦ **Pregunta 3: RBM Profundas y Convoluciones Desconocidas**

Considere dos desafíos avanzados en el aprendizaje profundo: la composición de modelos generativos y la implementación de una operación convolucional.

a) **Entrenamiento de Máquinas de Boltzmann Restringidas Profundas (DBNs) y Limitaciones:** Usted tiene un conjunto de datos muy complejo y desea modelarlo utilizando una **Red de Creencia Profunda (DBN)** apilando múltiples RBMs. Explique en detalle el proceso de **entrenamiento no supervisado capa por capa (greedy layer-wise pre-training)** de una DBN utilizando RBMs. Destaque por qué este enfoque secuencial es crucial para el entrenamiento exitoso de redes profundas generativas y cómo mitiga los problemas que surgirían al intentar entrenar la DBN de forma monolítica con Contrastive Divergence global. Discuta las limitaciones inherentes a este proceso, especialmente en términos de la optimización del modelo generativo final.

b) **Operación de Transposición Convoluta (Deconvolución o Transposed Convolution):** Una operación clave en arquitecturas generativas como los autoencoders variacionales (VAEs) y las redes generativas adversarias (GANs) es la **Transposición Convoluta** (a menudo incorrectamente llamada "deconvolución"). Dada una entrada de mapa de características de $H_{in} \times W_{in}$ y un filtro (kernel) de $K \times K$, con *stride* S y *padding* P , formule matemáticamente cómo se calcula la salida $H_{out} \times W_{out}$ de una operación de transposición convoluta. Proporcione la relación entre H_{in} , W_{in} , K , S , P y H_{out} , W_{out} . Ilustre con un ejemplo numérico concreto (p.ej., entrada 2×2 , filtro 3×3 , stride 1 , padding 0) mostrando cómo los elementos de la entrada contribuyen a la salida. Discuta brevemente por qué esta operación no es una "verdadera" deconvolución en el sentido matemático inverso, sino una convolución con un padding y stride específicos que emula un efecto de expansión.

▼ Solución

a) **Entrenamiento de Máquinas de Boltzmann Restringidas Profundas (DBNs) y Limitaciones**

Una **Red de Creencia Profunda (DBN)** es un modelo generativo profundo que se construye apilando múltiples Máquinas de Boltzmann Restringidas (RBMs) o combinando RBMs con capas superiores supervisadas.

Proceso de Entrenamiento No Supervisado Capa por Capa (Greedy Layer-wise Pre-training): El entrenamiento de una DBN se realiza en un proceso secuencial y codicioso:

1. **Entrenar la primera RBM:** La primera RBM se entrena de forma no supervisada con los datos de entrada brutos ($v_1 = x$). El objetivo es que esta RBM aprenda a extraer características de bajo nivel en su capa oculta (h_1). Se utiliza el algoritmo de **Contraste Divergente (CD)** para este entrenamiento.
2. **Entrenar la segunda RBM:** Una vez que la primera RBM está entrenada, sus pesos se congelan. Las activaciones de la capa oculta de la primera RBM (h_1) se utilizan como las entradas visibles para la segunda RBM ($v_2 = h_1$). Esta segunda RBM se entrena para aprender características de nivel superior (su capa oculta es h_2).
3. **Apilamiento Sucesivo:** Este proceso se repite para cada RBM en la pila. La capa oculta de una RBM entrenada se convierte en la capa visible de la siguiente RBM a entrenar. Así, cada RBM aprende una representación más abstracta y de mayor nivel de las características de la capa anterior.

Crucialidad del Enfoque Secuencial: Este enfoque es crucial por varias razones:

- **Mitigación de la Intratabilidad del Gradiente:** El entrenamiento de una DBN de forma monolítica (como una única RBM gigante) es computacionalmente intratable debido a la enorme suma sobre todas las configuraciones posibles en la función de partición. La aproximación de CD es efectiva para RBM individuales, pero no para la red profunda completa. El entrenamiento capa por capa divide un problema intocable en una serie de problemas manejables.
- **Manejo de la Complejidad del Paisaje de Pérdidas:** Para redes profundas, el paisaje de la función de pérdida es no convexo y altamente complejo, con muchos mínimos locales. El pre-entrenamiento no supervisado inicializa los pesos de la DBN en una región del espacio de parámetros que es más propicia para la optimización subsiguiente. Al aprender características significativas en cada capa, se proporciona una mejor "punto de partida" para el ajuste fino.
- **Aprendizaje de Representaciones Jerárquicas:** Cada RBM aprende a modelar las correlaciones en sus entradas, extrayendo características que son cada vez más abstractas y complejas a medida que se avanza en la pila. Esto permite a la DBN construir una jerarquía de características, similar a cómo el cerebro procesa la información visual (bordes, texturas, formas, objetos).

Limitaciones Inherentes: A pesar de sus ventajas, el pre-entrenamiento codicioso tiene limitaciones:

- **Óptimo Local en Cada Paso:** Cada RBM se entrena para ser óptima solo para modelar la capa de entrada inmediata, sin conocimiento de la tarea final ni de cómo sus representaciones impactarán las capas más profundas. Esto significa que la optimización de cada RBM es un óptimo local y no garantiza un óptimo global para la DBN completa.
- **Suboptimización Global:** Aunque el pre-entrenamiento proporciona una buena inicialización, la DBN resultante no es óptima para la tarea final (p. ej., clasificación) a menos que se realice un **ajuste fino (fine-tuning)** supervisado. El ajuste fino generalmente se hace agregando una capa de salida supervisada y usando backpropagation para ajustar todos los pesos de la DBN. Sin embargo, el ajuste fino aún puede quedarse atascado en un mínimo local.
- **Consumo de Recursos:** Aunque es más factible que el entrenamiento monolítico, entrenar múltiples RBMs secuencialmente sigue siendo computacionalmente intensivo, especialmente para datos muy grandes o RBMs muy amplias.

b) Operación de Transposición Convolutiva (Deconvolución o Transposed Convolution)

La **Transposición Convolutiva** es una operación fundamental en el *upsampling* de mapas de características en redes neuronales, usada comúnmente en autoencoders, GANs y segmentación semántica. No es una operación de "deconvolución" en el sentido de una inversa matemática de la convolución. Es una convolución estándar con un padding y/o stride particular que resulta en una expansión espacial del mapa de características.

Formulación Matemática y Relación entre Dimensiones: Dada una entrada $H_{in} \times W_{in}$, un kernel de tamaño $K \times K$, un *stride* S , y *padding* P . La operación de transposición convolutiva puede visualizarse de dos maneras:

1. **Convolución regular con padding expandido:** Se inserta $S - 1$ ceros entre cada elemento de la entrada y se aplica padding normal al borde exterior.
2. **La inversa de la operación de pooling/stride de una convolución:** Si una convolución mapea $H_{in} \times W_{in}$ a $H'_{out} \times W'_{out}$, la transposición convolutiva mapea $H'_{out} \times W'_{out}$ a $H_{in} \times W_{in}$.

Las dimensiones de salida $H_{out} \times W_{out}$ se calculan como:

$$H_{out} = S \cdot (H_{in} - 1) + K - 2P$$

$$W_{out} = S \cdot (W_{in} - 1) + K - 2P$$

(Esta fórmula asume un padding de P ceros alrededor del borde de la *entrada* expandida o la salida de la convolución inversa).

Ejemplo Numérico:

- **Entrada:** 2×2 (un solo canal)
- **Filtro:** 3×3 (un solo canal, pesos W_{ij})
- **Stride:** $S = 1$
- **Padding:** $P = 0$

Usando la fórmula: $H_{out} = 1 \cdot (2 - 1) + 3 - 2(0) = 1 + 3 = 4$

$W_{out} = 1 \cdot (2 - 1) + 3 - 2(0) = 1 + 3 = 4$ La salida esperada es 4×4 .

Ilustración del Cálculo (Conceptual): En una transposición convolucional, cada elemento de la entrada actúa como un "multiplicador" para el filtro, y el resultado de esa multiplicación se "suma" en la posición correspondiente de una matriz de salida más grande.

Considera una entrada I de 2×2 :

$$I = \begin{pmatrix} i_{00} & i_{01} \\ i_{10} & i_{11} \end{pmatrix}$$

Y un filtro K de 3×3 :

$$K = \begin{pmatrix} k_{00} & k_{01} & k_{02} \\ k_{10} & k_{11} & k_{12} \\ k_{20} & k_{21} & k_{22} \end{pmatrix}$$

La salida O de 4×4 se calcula de la siguiente manera: Cada i_{rc} de la entrada se multiplica por todo el filtro K , y el resultado se coloca en la matriz de salida O comenzando en la posición $(r \times S, c \times S)$ y sumándose a cualquier contribución existente.

- i_{00} se multiplica por K y se suma a $O[0 : 3, 0 : 3]$
- i_{01} se multiplica por K y se suma a $O[0 : 3, S : S + 3]$ (si $S = 1$, entonces $O[0 : 3, 1 : 4]$)
- i_{10} se multiplica por K y se suma a $O[S : S + 3, 0 : 3]$ (si $S = 1$, entonces $O[1 : 4, 0 : 3]$)
- i_{11} se multiplica por K y se suma a $O[S : S + 3, S : S + 3]$ (si $S = 1$, entonces $O[1 : 4, 1 : 4]$)

Por qué no es una "verdadera" deconvolución: Una verdadera deconvolución sería la operación inversa que recupera la entrada única que produjo una salida dada de una convolución, lo cual es matemáticamente problemático ya que la convolución es una operación de pérdida de información (muchas entradas diferentes pueden producir la misma salida). La transposición convolucional no es una inversa única. En cambio, es una operación que realiza una convolución en una entrada "extendida" o "padded" de tal manera que la salida tiene dimensiones mayores, útil para expandir mapas de características en el camino de un decodificador. Se le llama "transpuesta" porque la matriz que describe esta operación es la transpuesta de la matriz que describiría la convolución directa si se reformatearan las operaciones como multiplicaciones de matrices.

Contexto:

Una capa convolucional aplica filtros $\mathbf{K} \in \mathbb{R}^{k \times k}$ a una imagen $\mathbf{X} \in \mathbb{R}^{n \times n}$, produciendo una feature map $\mathbf{Y} \in \mathbb{R}^{m \times m}$.

Tarea:

1. Derive el gradiente $\frac{\partial \mathcal{L}}{\partial \mathbf{K}}$ dado $\frac{\partial \mathcal{L}}{\partial \mathbf{Y}}$.
2. Demuestre que la operación equivale a una convolución transpuesta (deconvolución).
3. Compare el número de parámetros de una capa convolucional vs. una densa para $n = 28, k = 5, m = 24$.

Ejemplo: Para $\mathbf{X} \in \mathbb{R}^{3 \times 3}$ y $\mathbf{K} \in \mathbb{R}^{2 \times 2}$, calcule $\frac{\partial \mathcal{L}}{\partial \mathbf{K}}$ dado $\frac{\partial \mathcal{L}}{\partial \mathbf{Y}} \in \mathbb{R}^{2 \times 2}$.

Solución

1. Derivación del gradiente $\frac{\partial \mathcal{L}}{\partial \mathbf{K}}$

La salida de la capa convolucional se define como

$$Y_{i,j} = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} K_{u,v} X_{i+u, j+v}$$

para $i, j = 0, \dots, m-1$.

Si denotamos $G_{i,j} = \frac{\partial \mathcal{L}}{\partial Y_{i,j}}$, entonces

$$\frac{\partial \mathcal{L}}{\partial K_{u,v}} = \sum_{i=0}^{m-1} \sum_{j=0}^{m-1} G_{i,j} \frac{\partial Y_{i,j}}{\partial K_{u,v}} = \sum_{i=0}^{m-1} \sum_{j=0}^{m-1} G_{i,j} X_{i+u, j+v}.$$

En forma matricial, esto equivale a correlacionar la entrada $\mathbf{X}$ con el gradiente $\mathbf{G}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{K}} = \mathbf{X} \star \mathbf{G},$$

donde ($\star$) denota correlación válida (sin relleno).

2. Equivalencia con la convolución transpuesta

Al reorganizar los sumatorios, observamos que

$$(\mathbf{X} \star \mathbf{G})_{u,v} = \sum_{i,j} X_{i+u, j+v} G_{i,j} = \left(\text{rot180}(\mathbf{G}) * \mathbf{X} \right)_{u,v},$$

es decir, una convolución de $\mathbf{X}$ con $\mathbf{G}$ rotada 180° .

Esta operación coincide con la definición de convolución transpuesta (deconvolución), donde el gradiente sobre los pesos se calcula aplicando una convolución "inversa" de los gradientes de la salida sobre la entrada original.

3. Comparación de parámetros: capa convolucional vs capa densa

Para una imagen $\mathbf{X} \in \mathbb{R}^{28 \times 28}$, filtro de tamaño $k = 5$ y output $\mathbf{Y} \in \mathbb{R}^{24 \times 24}$:

Tipo de capa	Número de parámetros
Convolucional (1 filtro, sin sesgo)	$k \times k = 5 \times 5 = 25$
	$n^2 \times m^2 = 28^2$
Densa (conexión completa)	$\times 24^2 = 784 \times 576$
	$= 451,584$

4. Ejemplo concreto

Sea $\mathbf{X} \in \mathbb{R}^{3 \times 3}$, $\mathbf{K} \in \mathbb{R}^{2 \times 2}$ y $\mathbf{G} = [G_{i,j}] \in \mathbb{R}^{2 \times 2}$, donde $G_{0,0}, G_{0,1}, G_{1,0}, G_{1,1}$ son los componentes de $\partial \mathcal{L} / \partial \mathbf{Y}$.

Los componentes de $\partial \mathcal{L} / \partial \mathbf{K}$ son:

$\frac{\partial \mathcal{L}}{\partial K_{u,v}}$	Expresión
$u = 0, v = 0$	$G_{0,0} X_{0,0}$ $+ G_{0,1} X_{0,1}$ $+ G_{1,0} X_{1,0}$ $+ G_{1,1} X_{1,1}$
$u = 0, v = 1$	$G_{0,0} X_{0,1}$ $+ G_{0,1} X_{0,2}$ $+ G_{1,0} X_{1,1}$ $+ G_{1,1} X_{1,2}$
$u = 1, v = 0$	$G_{0,0} X_{1,0}$ $+ G_{0,1} X_{1,1}$ $+ G_{1,0} X_{2,0}$ $+ G_{1,1} X_{2,1}$
$u = 1, v = 1$	$G_{0,0} X_{1,1}$ $+ G_{0,1} X_{1,2}$ $+ G_{1,0} X_{2,1}$ $+ G_{1,1} X_{2,2}$

Con esto, queda demostrado el cálculo explícito del gradiente sobre cada componente de $\mathbf{K}$.